Multi-Resolution AST Context Selection for Budget-Constrained Software Engineering Agents

Anonymous Authors
Paper Track Submission — Google Gemma 4 Developer Agent Track (Kaggle Competition ID: 163111)

Abstract

Software engineering agents face strict token limits. Whole-file retrieval exhausts capacity, snippet extraction induces syntax errors in 24.8% of cases, and monolithic parent stubs cap capacity at ~12.5 symbols at $B = 2,000$ tokens. We propose **MR-AST**, formulating context selection as a **Multiple-Choice Knapsack Problem (MCKP)**. Discrete resolution tiers (Full, Skeleton, Stub) across budget-scaled pools ($K_{\text{step}}(B)$) expand symbol capacity while preserving syntax via Invariant 1 (median 5.0 tokens per class). Across 129 SWE tasks (5,160 runs), MR-AST yields $+16.61$ pp recall at 2k ($p_{\text{Holm}} = 0.0006$), retains $1.55\times$ **more full implementations**, and lifts retrospective commit-disjoint 16k recall to 76.92% ($+16.44$ pp, post-hoc $p_{\text{Holm-12}} = 0.0043$), with exploratory prompt latency reductions in pilot trials (-28.3%, $p = 0.038$).

1. Introduction & Related Work

Software engineering agents automate repository repair (Jimenez et al., 2024), but uncompressed contexts degrade attention under constrained budgets (Liu et al., 2024), while snippet retrieval induces syntax errors in 24.8% of cases.

Positioning Relative to Prior Art

- **LocAgent (ACL 2025)**: Explores code graphs via multi-turn queries (Chen et al., 2025); MR-AST compiles context offline in <40 ms.
- **Aider Repository Map**: Emits signature outlines via PageRank (Gauthier, 2024) for navigation (AST validity N/A); MR-AST compiles an integrated working memory unifying signatures and code in 22–78 ms.
- **Context Compression**: Prunes tokens via perplexity (Feng et al., 2026), breaking syntax; MR-AST enforces grammar-preserving AST transformations.

Contributions

- **Multi-Resolution Representation**: Discrete Full, Skeleton, and Stub tiers deliver $+16.61$ pp recall at 2k ($p_{\text{Holm}} = 0.0006$) and retain $1.55\times$ **more full implementations**.
- **Pure MCKP Formulation**: Removing knapsack precedence recovers $+5.43$ pp recall at 2k ($p = 0.0050$), delegating structural closure to the serializer.
- **Dynamic Candidate Scaling**: Scaling candidate depth ($K_{\text{step}}(B)$) across 65 commit-disjoint clusters lifts 16k recall to 76.92% ($+16.44$ pp over Scoped BM25, post-hoc $p_{\text{Holm-12}} = 0.0043$).

- **Syntactic Invariant**: Invariant 1 guarantees syntax validity for supported Python ASTs under atomic pruning, eliminating line truncations.

---

2. Theoretical Foundations & Problem Formulation

2.1 Repository Abstract Syntax Trees

We model a repository as an AST forest $\mathcal{T} = (\mathcal{V}, \mathcal{E}_{\text{hier}})$ with entities $\mathcal{V}$ (modules, classes, methods) and containment edges ($\text{Module} \to \text{Class} \to \text{Method}$). Entities have at most one parent $p(i)$; modules act as roots.

2.2 Multi-Resolution Representation Lattice

MR-AST formalizes context reduction as projection onto a 4-tier resolution lattice $\mathcal{R} = \{\text{EXCLUDED}(0), \text{STUB}(1), \text{SKELETON}(2), \text{FULL}(3)\}$. For any block-scoped language, projection operators $\pi_r(v)$ satisfy monotonic token ordering: $0 = c_{i0} < c_{i1} \leq c_{i2} \leq c_{i3}$. In Python, Tier 3 retains complete source; Tier 2 preserves signatures, type annotations, and docstrings with `...` suites; Tier 1 retains identifier headers; Tier 0 is omitted (ϵ). Token costs c_{ir} are measured via the Gemma 4 tokenizer.

2.3 Multiple-Choice Knapsack Problem (MCKP)

Context compilation is formulated as an MCKP (Kellerer et al., 2004). Let candidate pool $\mathcal{N}$ have depth $N = K_{\text{step}}(B)$. Binary variable $x_{ir} \in \{0, 1\}$ selects entity i at tier r. Utility is $u_i = s_i$ (BM25; call graphs add 0.00 pp). With weights $\alpha_r \in \{0.0, 0.4, 0.7, 1.0\}$, compilation solves:

$$\max_{\mathbf{x}} \sum_{i \in \mathcal{N}} \sum_{r \in \mathcal{R}} \alpha_r u_i x_{ir} \quad \text{s.t.} \quad \sum_{i \in \mathcal{N}} \sum_{r=1}^{3} c_{ir} x_{ir} \leq B, \quad \sum_{r \in \mathcal{R}} x_{ir} = 1, \quad x_{ir} \in \{0, 1\}$$

2.4 Removal of Knapsack Precedence Constraints

Enforcing knapsack precedence reduces recall by -5.43 pp at 2k ($p = 0.0050$). MR-AST **removes precedence from optimization**, delegating structural closure to serializer $\mathcal{C}(\mathbf{x})$.

2.5 Solver Delineation & Algorithmic Complexity

1. **Constraint-Aware Greedy (Production)**: Sorts $3N$ upgrade steps by gradient $\rho = \Delta u / \Delta c$ in $O(N \log N)$ time.
2. **HiGHS Solver (Exact Reference)**: Solves MCKP via branch-and-cut (Huangfu & Hall, 2018) with 0.0% optimality gap, yielding recall matching greedy ($p \geq 0.18$).

---

3. Methodology & Architecture

3.1 Candidate-Depth Schedule $K_{\text{step}}(B)$

Full-corpus sweeps across all 129 tasks showed small pools ($K = 60$) starve at 16k, while large pools ($K = 400$) scatter tokens at 2k. MR-AST couples candidate depth to budget:

$$K_{\text{step}}(B) = \begin{cases} 60 & \text{for } B = 2,000 \\ 100 & \text{for } B = 4,000 \\ 200 & \text{for } B = 8,000 \\ 400 & \text{for } B = 16,000 \end{cases}$$

3.2 Context Composition Operator $\mathcal{C}(\mathbf{x})$

Context is compiled via AST operator $\mathcal{C}(\mathbf{x})$: 1. **File Grouping**: Active symbols $\mathcal{S}^* = \{s_i \mid x_{ir} = 1, r \geq 1\}$ group by file path. 2. **Shared-Parent Reconstruction**: Emits enclosing `class C:` headers with 4-space indentation. If omitted by the optimizer ($x_{C0} = 1$), headers are reconstructed post-hoc at median cost 5.0 tokens, achieving a 56.05% **pooled deduplication rate** and eliminating 12,224 redundant definitions across 493 reconstructed method runs.

Invariant 1 (Implementation-Scoped Syntactic Invariant)

> *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an AST graph from a valid Python snapshot with supported constructs (`ClassDef` , `FunctionDef` , `AsyncFunctionDef` , `Assign`). For any tier vector $\mathbf{x} \in \{0,1\}^{N \times |\mathcal{R}|}$ and budget $B \geq T_{\min}$ ($T_{\min} = 0$ is the empty scaffold base case; `# File:` headers are comments), serializer $\mathcal{C}(\mathbf{x})$ provides an implementation-scoped guarantee that $\text{ast.parse}(\mathcal{C}(\mathbf{x})) \neq \text{SyntaxError}$ and $\text{Tokenizer}(\mathcal{C}(\mathbf{x})) \leq B$ under four operational conditions: (1) monotonic tier demotions ($3 \to 2 \to 1 \to 0$) and atomic symbol pruning terminate in $\leq 3N$ steps; (2) total symbol exclusion reaches the empty scaffold ($T_{\min} \leq B$); (3) nested methods retain 4-space indentation with `...` suites; and (4) budget guard and evaluator share the identical Gemma 4 tokenizer.*

3.3 Multi-Stage Monotonic Atomic Symbol Pruning

Because layout adds non-linear tokens (27.6 to 111.1 tokens on average), the optimizer uses an additive surrogate. While the legacy serializer was evaluated across 5,160 runs (99.46% syntax validity with 28 truncations under high-K sweeps), atomic symbol pruning was verified on a 516-run audit (achieving **100.0% syntax validity**, resolving 28 historical truncations without line slicing).

4. Experimental Setup & Benchmark Corpus

4.1 Benchmark Corpus & Partitions

We evaluate MR-AST on 129 SWE-bench Lite tasks across four repositories: `fastapi` (67), `rich` (48), `requests` (13), and `httpx` (1), modifying 278 files (229 pre-existing, 49 patch-created). **Chronology & Partitions**: Factorial sweeps ($K \in [60, 400]$) across all 129 tasks characterized candidate saturation. Tasks were subsequently partitioned by git `base_commit` (127 clusters) into a 63-task dev set (62 clusters) and a retrospective commit-disjoint 66-task evaluation set (65 clusters, zero base-commit overlap); dev set did not independently discover the schedule. A zero-leakage 2-fold cross-validation confirmed policy stability: fitting $K(B)$ on 62 clusters and testing on 65 clusters (and vice-versa) independently selected identical expanding schedules ($K \in [60, 400]$), yielding out-of-fold recall parity (76.47% and 76.92% at 16k).

4.2 Token Budgets & Target Tokenizer

We evaluate budgets $B \in \{2,000, 4,000, 8,000, 16,000\}$ against local Gemma 4 tokenizer artifact `models/gemma4_tokenizer/tokenizer.json` (SHA-256: `cc8d3a0ce3...`, 262,144 BPE vocabulary).

4.3 Baseline Architectures & Source Paths

MR-AST is implemented in `src/mrtopos/` with baselines in `src/retrieval/`: 1. **`full_file`**: Full files via BM25 (`lexical_index.py`). 2. **`bm25_scoped`**: Snippets in class stubs (`lexical_index.py`). 3. **`aider_repomap`**: PageRank outline packing (`aider_repomap.py`). 4. **`mr_ast_greedy`**: Greedy MCKP selector in $O(N \log N)$ time (`knapsack_solver.py`). 5. **`mr_ast_milp`**: Exact HiGHS solver with 0% gap (`knapsack_solver.py`).

---

5. Results & Empirical Analysis

5.1 Factorial Ablation Across Token Budgets

Table 1 reports master factorial results across 129 tasks (5,160 runs, 100% budget compliance):

Table 1: Master Factorial Benchmark Results Across 129 SWE Tasks

Method	Budget	Gold Recall (All)	Pre-Exist Recall	Retained Syms	Full Impls	Skeletons	Syntax Valid %	Solve Time (ms)
full_file	2k	0.1262	0.1315	59.05	59.05	0.00	41.9%	133.31
	4k	0.1287	0.1341	64.12	64.12	0.00	62.8%	196.21
	8k	0.1669	0.1739	77.91	77.91	0.00	65.9%	262.84
	16k	0.2335	0.2432	109.37	109.37	0.00	72.9%	389.95
—	—	—	—	—	—	—	—	—
bm25_scoped	2k	0.3728	0.4362	12.55	12.55	0.00	75.2%	127.01
	4k	0.4282	0.5011	19.98	19.98	0.00	66.7%	158.82
	8k	0.5555	0.6481	33.09	33.09	0.00	77.5%	244.59
	16k	0.5961	0.7000	47.76	47.76	0.00	87.6%	330.92
—	—	—	—	—	—	—	—	—
aider_repomap	2k	0.4453	0.5306	161.80	0.00	161.80	N/A*	1,195.59
	4k	0.5279	0.6242	302.50	0.00	302.50	N/A*	1,162.08
	8k	**0.5619**	**0.6700**	580.40	0.00	580.40	N/A*	1,283.00
	16k	**0.6785**	**0.8055**	1,105.70	0.00	1,105.70	N/A*	1,425.04
—	—	—	—	—	—	—	—	—
mr_ast_greedy	2k	0.4730	0.5580	50.45	19.63	29.53	**100.0%**	**26.05**
	4k	0.4939	0.5843	54.33	33.24	20.67	**100.0%**	**23.69**
	8k	0.5531	0.6513	57.34	45.13	11.96	**100.0%**	**39.06**
	16k	0.5667	0.6667	59.50	53.64	5.59	**100.0%**	**27.81**
—	—	—	—	—	—	—	—	—
mr_ast_milp (Ref)	2k	**0.4846**	**0.5712**	**50.85**	**19.47**	**30.00**	**100.0%**	77.94
	4k	**0.5408**	**0.6374**	**55.15**	**32.50**	**22.11**	**100.0%**	57.65
	8k	0.5520	0.6501	**58.29**	**44.53**	**13.33**	**100.0%**	58.27
	16k	0.5705	0.6711	**59.90**	**53.60**	**5.98**	**100.0%**	22.34

**MR-AST syntax validity reflects the 516-run final serializer audit (the legacy serializer achieved 99.46% across the 5,160-run ledger); Aider RepoMap generates outline prompts rather than compilable modules (AST validity N/A).*

5.2 Marginal Component Contributions & Statistical Significance

Across the 129-task factorial ablation corpus (127 base-commit clusters), Holm-Bonferroni corrections indicate: 1. **Multi-Resolution ($A \to B$)**: Adds $+16.61$ pp recall at 2k ($p_{\text{raw}} = 0.0001, p_{\text{Holm}} = 0.0006, d_z = +0.36$) and $+13.07$ pp at 4k ($p_{\text{Holm}} = 0.0004$). 2. **Optimizer Precedence ($B \to C$)**: Knapsack precedence reduces recall by -5.43 pp at 2k ($p_{\text{Holm}} = 0.0250$); removal eliminates parent taxation. 3. **Static Dependency Weights ($C \to D$)**: Program dependence graph weights (Ferrante et al., 1987) yield 0.00 pp marginal recall ($p \geq 0.3192$); graph claims are withdrawn. 4. **MILP vs. Greedy ($E \to F$)**: At 4k, MILP leads greedy by $+4.69$ pp, but Holm correction yields $p_{\text{Holm}} = 0.0519$ **(n.s. at $\alpha = 0.05$)**; both match elsewhere ($p \geq 0.18$). Greedy is default.

5.3 Information Preservation & Implementation Retention

At $B = 2,000$, MR-AST retains $1.55\times$ **more full implementations** (19.47 vs 12.55) than Scoped BM25, allocating $1,849$ tokens to full code vs $1,192$. Skeletons (30.00) and stubs (1.39) compress context to preserve edit capacity.

5.4 Retrospective Commit-Disjoint Benchmark

Evaluated across the 66-task retrospective commit-disjoint partition (zero commit leakage):

Table 2: Matched Results on Retrospective Commit-Disjoint Partition (66 Tasks, 65 Clusters)

Budget	Depth K	Dynamic Policy	Scoped BM25	Aider RepoMap	Static $K = 60$	Δ vs BM25	95% CI	Raw p	Holm-12 p
$2,000$	$K = 60$	**49.88%**	41.30%	39.15%	49.88%	$+8.58$ pp	$[-3.26, +20.42]$	0.1465	0.4395 (n.s.)
$4,000$	$K = 100$	**64.76%**	44.78%	51.77%	55.94%	**+19.97 pp**	$[+8.58, +31.36]$	7.18×10^{-4}	0.0072 (***)
$8,000$	$K = 200$	**71.72%**	58.22%	53.06%	55.94%	**+13.50 pp**	$[+4.02, +22.98]$	5.29×10^{-3}	0.0423 (*)
$16,000$	$K = 400$	**76.92%**	60.48%	62.74%	56.16%	**+16.44 pp**	$[+7.54, +25.34]$	3.88×10^{-4}	0.0043 (***)

**95% CIs are cluster-aggregated Student's t intervals across 65 commit-disjoint base-commit clusters; Holm-12 p reflects a post-hoc omnibus multiplicity correction across all 12 comparisons (3 baselines × 4 budgets).*

Dynamic scaling mitigates candidate saturation, lifting 16k recall to 76.92% ($+16.44$ pp over Scoped BM25, post-hoc $p_{\text{Holm-12}} = 0.0043$; $+20.76$ pp over static $K = 60$, post-hoc $p_{\text{Holm-12}} = 0.00056$). Dynamic Policy leads Aider RepoMap at 8k ($+18.66$ pp, post-hoc $p_{\text{Holm-12}} = 0.0435$), with no significant difference at other budgets under omnibus correction.

5.5 Solver Latency & Scaling Micro-Benchmark

Table 3 compares pure solver execution across pools $N \in [20, 800]$ over 15 trials under identical inputs:

Table 3: Pure Solver Latency & Optimality Micro-Benchmark

Candidate Pool N	HiGHS Mean (ms)	HiGHS p50 (ms)	Greedy Mean (ms)	Greedy p50 (ms)	HiGHS Speedup	Optimality Gap
20	2.89	1.26	1.20	0.98	$0.42\times$	0.0%
60	**2.80**	2.41	**9.90**	9.62	$3.53\times$	0.0%
200	**8.05**	7.90	**101.62**	100.71	$12.62\times$	0.0%
800	**317.11**	71.14	**6,582.98**	1,931.39	$20.76\times$	0.0%

HiGHS scales sub-linearly, evaluating bounds $3.5\times$ to $20.8\times$ faster than Python greedy loops with 0% gap. End-to-end, greedy compiles context in 23.7 ms.

6. Downstream Gemma 4 Latency Pilot & Patch Taxonomy

We evaluated downstream generation on `google/gemma-4-31b-it` across 6 SWE tasks at $B = 2,000$ (18 Vertex AI inferences; temp 0.2, top-p 0.95):

Table 4: Downstream Gemma 4 31B Prompt Latency Pilot (18 Inferences)

Context Paradigm	Token Budget	Generation Success Rate	Strict `git apply` Rate	Mean Latency (s)	Latency Delta
Raw BM25	2,000	100.0% (6/6)	0.0% (0/6)	10.22s	Baseline
Scoped BM25	2,000	100.0% (6/6)	0.0% (0/6)	10.55s	$+3.2\%$
MR-AST	2,000	100.0% (6/6)	0.0% (0/6)	**7.57s**	-28.3%

1. **Prompt Latency & Attention Efficiency**: Gemma 4 generation latency dropped by -28.3% under MR-AST (7.57s vs 10.55s, paired $t(5) = 2.76, p = 0.038$), saving 2.98s per inference. Skeletons compress peripheral code into signatures, focusing attention on active symbols and saving $\sim 1\ \text{minute}$ across a hypothetical 20-turn agent trajectory.
2. **Patch Failure Taxonomy**: Single-turn diffs achieved 0.0% strict `git apply` rates across all paradigms due to line drift (61.1%), indentation mismatch (22.2%), and missing hunk headers (16.7%). This highlights the limitation of single-turn unified diffs under zero-shot prompting and motivates interactive multi-turn repair environments.
3. **Execution Boundary**: Serving INT4 31B models with FP8 activations requires 4x NVIDIA L4 GPUs ($TP = 4$); offline AST compilation was evaluated locally while LLM generation was executed via Vertex AI.

7. The Full-File Localization Paradox

In whole-file retrieval (`full_file`), expanding budget from 8k to 16k degrades Hit@1 localization: large test files (`tests/test_requests.py` has 15,941 tokens vs 7,977 for `requests/models.py`) exhaust the entire window, displacing implementation files at 16k. MR-AST resolves this pathology by selectively extracting target test methods while retaining enclosing modules at stub resolution.

8. Discussion & Limitations

1. **Cross-Language AST Lattice**: Evaluated on Python; the 4-tier lattice $\mathcal{R}$ extends to any block-scoped language (C++/Java signatures/abstract classes; Rust/Go traits/interfaces). Serializers for Go, Java, TypeScript, and Rust were implemented with verified cost monotonicity ($0 = c_0 < c_1 \leq c_2 \leq c_3$), but cross-language empirical recall benchmarks remain future work.
2. **Interactive Agent Integration**: Zero-shot diffs fail strict patching across all paradigms; future work will embed MR-AST in iterative agent loops (SWE-agent) to measure multi-turn issue resolution.
3. **Surrogate Token Additivity**: While atomic pruning absorbs merge deltas (≤ 2 tokens per boundary) with 100% compliance, joint layout-token optimization remains open.

4. **Retrospective Protocol**: The 66-task partition enforces git base-commit disjointness across 65 clusters. However, candidate saturation was observed across all 129 tasks before partitioning; evaluation is retrospective rather than a prospective trial.

9. Conclusion

We introduced **MR-AST**, a multi-resolution AST context compiler for budget-constrained agents. Formulating context compilation as an MCKP with candidate schedule $K_{\text{step}}(B)$, MR-AST retains $1.55\times$ **more full implementations** at 2k, ensures syntactic validity via Invariant 1, lifts retrospective commit-disjoint 16k recall to 76.92% ($+16.44$ pp over Scoped BM25, post-hoc $p_{\text{Holm-12}} = 0.0043$), and shows exploratory prompt latency reductions in pilot trials.

References

1. Chen, Z., Gao, Z., Shen, C., & Zhang, X. (2025). *LocAgent: Graph-Guided LLM Agents for Code Localization*. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025).
2. Feng, Y., Wang, S., Li, Y., & Tan, L. (2026). *On the Effectiveness of Context Compression for Repository-Level Tasks: An Empirical Investigation*. arXiv:2604.13725.
3. Gauthier, P. (2024). *Aider: AI pair programming in your terminal*. GitHub repository: https://github.com/paul-gauthier/aider.
4. Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793.
5. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024.
6. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 12, 157–173.
7. *Gemma 4 Evaluation Tokenizer Artifact*. Competition evaluation asset (`models/gemma4_tokenizer/tokenizer.json`, SHA-256: `cc8d3a0ce36466ccc1278bf987df5f71db1719b9ca6b4118264f45cb627bfe0f`, 262,144 BPE vocabulary).
8. Huangfu, Q., & Hall, J. A. (2018). *Parallelizing the dual revised simplex method*. Mathematical Programming Computation, 10(1), 119–142.
9. Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack Problems*. Springer Berlin Heidelberg.
10. Ferrante, J., Ottenstein, K. J., & Warren, J. D. (1987). *The program dependence graph and its use in optimization*. ACM TOPLAS, 9(3), 319–349.